



Desafío I

Sweet Crush con manipulación de bits en C++

Luis Miguel Cohen Teherán

Informática II, semestre 2026-2

Universidad de Antioquia
Facultad de Ingeniería
2026

1. Introducción

El Desafío I de Informática II (2026-2) pide un Sweet Crush en C++, en un proyecto Qt de consola y con programación estructurada. El tablero es de F filas y C columnas, valores que el usuario escribe al iniciar. Cada casilla ocupa tres bits en un bloque continuo de unsigned char pedido con new[]. El tamaño del bloque es $(3 \cdot F \cdot C + 7) / 8$ bytes, con división entera, y las fichas se generan al azar entre seis tipos. No se reserva un byte ni un entero por posición, ni se deja el tablero desempquetado en una matriz aparte.

El usuario no intercambia fichas. En cada turno el menú admite una sola operación: borrar la ficha de una coordenada, insertar o eliminar una fila, insertar o eliminar una columna, o salir. Al borrar una ficha, esa casilla queda en 110, la columna desciende y arriba entran fichas nuevas. Después se buscan tres o más iguales seguidas, en horizontal o en vertical; si hay, se marcan todas, se eliminan, cae el tablero completo y se vuelve a buscar. Ese ciclo es la cascada. Insertar una fila o una columna siempre pide un bloque nuevo y copia ficha a ficha. Eliminar compacta sobre el mismo arreglo y solo pide un bloque más pequeño cuando la ocupación baja de 0.65 respecto a la última reserva. Después de cada jugada salen la estadística, la trama en binario del bit 7 al 0 de cada byte, y el tablero con símbolos: dimensiones, bytes, ocupación, eliminaciones del usuario, combinaciones, cascadas, fichas y puntaje. Cada combinación suma un punto; las fichas van en otro contador. La consola no se borra entre turnos. Cada bloque se marca como TABLERO INICIAL o JUGADA n, con un renglón de lo que se hizo.

En el proyecto no se permitió el uso de struct, class, template, string ni la STL, y tampoco malloc ni printf. Fuera de bits.h y tablero.h se usan iostream, para cin y cout, y cstdlib con ctime, porque el azar va con srand(time(nullptr)) una vez al inicio y rand()%6. El repositorio se fue subiendo por avances, como pedían los commits periódicos.

1.1 Objetivo general

Implementar Sweet Crush en C++ como aplicación Qt de consola, almacenando el tablero en un bloque continuo de unsigned char a tres bits por casilla, con memoria dinámica y programación estructurada.

1.2 Objetivos específicos

1. Leer y escribir cada ficha con operadores de bits, incluido el caso en que el grupo de tres bits queda partido entre dos bytes consecutivos.

2. Reservar el número mínimo de bytes del tablero, $(3 \cdot F \cdot C + 7) / 8$, y mostrar después de cada jugada la trama binaria, el tablero con símbolos y los indicadores de tamaño, eliminaciones, combinaciones, cascadas y puntaje.

3. Detectar de manera simultánea tramos de tres o más fichas iguales en horizontal y en vertical, aplicar gravedad, resolver las cascadas y asignar un punto por combinación, de modo que una T o una L cuenten como una sola.

4. Insertar y eliminar una fila o una columna en cualquier posición, copiando ficha a ficha: al insertar se pide siempre un bloque nuevo; al eliminar se compacta en el mismo arreglo y solo se reduce la reserva si la ocupación baja del 65%.

5. Ofrecer en el menú una sola operación por turno y omitir las acciones que no aplican cuando el tablero queda sin filas, sin columnas o sin casillas.

2. Análisis del problema

2.1 Representación de las fichas

Con tres bits salen ocho valores. Seis son las fichas y quedan dos libres. La Tabla 1 es la que se usó. Vacía es 110, o sea 6, mientras cae una columna o se borra un grupo; con el tablero quieto no debería verse. El 111 no se escribe. Otra tabla válida era 0 para vacía y 1 a 6 para las fichas: también cabe en tres bits. No se usó porque `rand()%6` ya da 0 a 5 y no hay que sumarle uno al código. En consola cada tipo lleva un carácter. string no se puede, así que los seis símbolos van en un arreglo de char de tamaño 6.

Tabla 1. Codificación de 3 bits y símbolos de consola.

Valor	Bits	Significado	Símbolo
0	000	ficha	#
1	001	ficha	@
2	010	ficha	O
3	011	ficha	X
4	100	ficha	+
5	101	ficha	%
6	110	casilla vacía	.
7	111	no usado	

2.2 Mapeo de una casilla a la trama de bits

El tablero se recorre por filas, de izquierda a derecha y de arriba hacia abajo. Para una casilla en la fila f y la columna c , el índice es $i = f \cdot C + c$ y el primer bit de esa ficha está en $b = 3 \cdot i$, contado desde el bit menos significativo del byte 0. Las reglas de memoria piden que la trama válida empiece ahí y no al otro lado del byte. El byte es $b/8$ y el offset es b módulo 8. Como 3 no divide a 8, los tres bits caben en un solo byte si $\text{offset} + 3$ no pasa de 8, o se parten: lo que alcanza en el byte actual se toma desde el offset y el resto desde el LSB del siguiente. Si se corre sin recortar con 7, se mezclan bits de la ficha de al lado.

Cuando $3 \cdot F \cdot C$ no es múltiplo de 8 sobran entre 1 y 7 bits. No pueden quedar en el medio de la trama; hay que agruparlos a la izquierda, en los de mayor peso del último byte. El tamaño mínimo es $(3 \cdot F \cdot C + 7)/8$. El +7 hace el techo con división entera, sin float. Un 10×10 son 300 bits, 38 bytes. Un 2×3 son 18 bits y 3 bytes; la ficha de índice 2 empieza en el bit 6 y se parte, como en la Figura 1.

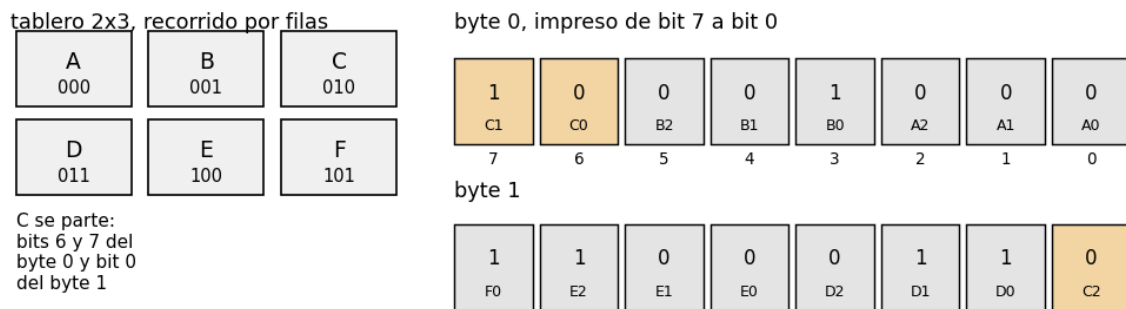


Figura 1. Empaquetado de un tablero 2x3 desde el LSB. La ficha C ocupa los bits 6 y 7 del byte 0 y el bit 0 del byte 1.

2.3 Consideraciones de la alternativa

Se pensó en copiar el tablero a un arreglo de enteros, resolver ahí las combinaciones y compactar al final. Es más cómodo, pero no se puede dejar una representación permanente en la que cada ficha ocupe un byte o un int. Por eso leerFicha y escribirFicha son el único acceso a una casilla. Caer, combinar y cambiar el tamaño pasan por esas dos.

La primera reserva es justa: `new unsigned char[bytesDelTablero(F,C)]`. Se podía pedir de una vez un bloque grande para no volver a reservar. No se puede dejar un tablero sobredimensionado ni memoria para expansiones futuras. Si F o C crecen, el bloque anterior no alcanza y hay que hacer `new[]`. Al copiar no se usa `memcpy` del arreglo de bytes: si la fila o la columna entra en el medio, los grupos de 3 bits se corren y una ficha partida ya no cae en el mismo offset. Se copia con `leerFicha` y `escribirFicha`.

Otra salida al quitar era pedir siempre un bloque nuevo y copiar todo. Eso corre y no rompe las reglas de memoria. No se dejó como único camino porque en un 10×10, al pasar de 100 a 90 celdas, la ocupación sigue en 90% y no hace falta otro new. Se compacta en el mismo arreglo y revisarMemoria solo reserva de nuevo si usadas*100 < reserva*65.

Hay tres tipos de eliminación y no se mezclan. Borrar una coordenada es de menú y cuenta como eliminación del usuario. Las tiras de tres o más iguales se quitan solas. Quitar una fila o una columna también es de menú y es lo único que cambia F o C. Cascada es la caída que sigue a borrar combinaciones, aunque no se arme otro grupo. Quitar una columna no es cascada: las fichas no bajan, se pegan. El puntaje queda a criterio. Se podía dar recargo por T o L. No se hizo, el cruce es un solo grupo de marcas, así que vale un punto, igual que una tira de tres. Dos tiras que no se tocan sí son dos. No se imprimen los tableros de en medio. srand va una sola vez al iniciar. Si se llama dentro de fichaAlAzar, varias fichas en el mismo segundo salen iguales. rand()%6 da 0 a 5, uniforme entre los seis tipos.

3. Arquitectura modular

SweetCrush.pro deja consola, sin módulo gui. Se podía armar ventana. No se hizo, la entrada y la salida son cin y cout, y un widget no aporta al empaquetado. El código está en bits.h y bits.cpp (empaquetado y azar), tablero.h y tablero.cpp (reserva, vistas, caída, combinaciones y tamaño) y main.cpp (F, C, menú e indicadores). En el diseño preliminar iba a haber un archivo para la partida. No se hizo: el menú solo llama a tablero.h y no guarda un estado aparte. Las cabeceras ajenas son iostream, cstdlib y ctime.

SweetCrush.pro · aplicacion de consola, sin modulo gui

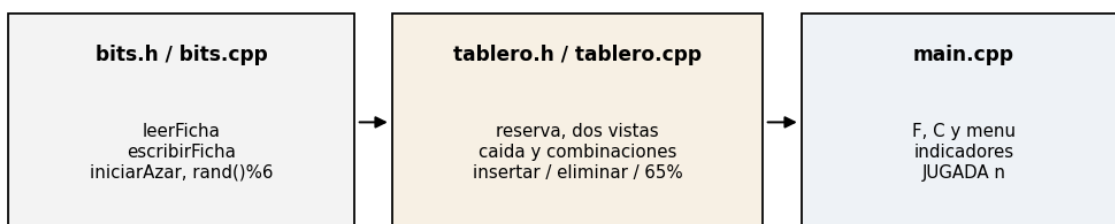


Figura 2. Partición del código: empaquetado, tablero y menú.

3.1 SweetCrush.pro

SweetCrush.pro es el archivo de proyecto de Qt. QT -= gui y CONFIG += console cmdline dejan una aplicación de terminal: cin y cout van a la consola, no a un widget. HEADERS enumera bits.h y

tablero.h; SOURCES enumera main.cpp, bits.cpp y tablero.cpp. No hay más pares de archivos. Al abrir ese .pro en Qt Creator se obtiene el ejecutable que se usa en la demostración, siempre con Run in terminal.

3.2 bits.h y bits.cpp

bits.h declara el acceso a una casilla y las constantes de los ocho códigos. FICHA_MIN y FICHA_MAX valen 0 y 5. FICHA_VACIA vale 6, el patrón 110, y FICHA_NO_USADA vale 7, que no se escribe. Ahí quedan también iniciarAzar, fichaAlAzar, leerFicha y escribirFicha. El encabezado no conoce F ni C: trabaja con un índice lineal y con el puntero al bloque. La guarda BITS_H evita una inclusión doble.

bits.cpp incluye bits.h, cstdlib y ctime. No usa iostream. iniciarAzar llama srand(time(nullptr)) una vez. fichaAlAzar devuelve rand()%6. leerFicha y escribirFicha calculan bitInicio = indice*3, el byte y el offset, y cubren el caso en que la ficha cabe en un byte y el caso en que se parte. El resto del programa no enmascara bits por su cuenta: si una función de tablero.cpp necesita una casilla, llama a estas dos.

3.3 tablero.h y tablero.cpp

tablero.h declara las operaciones sobre el bloque completo. bytesDelTablero, crearTablero y liberarTablero piden y sueltan la memoria. mostrarBinario y mostrarTablero son las dos vistas obligatorias. borrarFicha deja 110 en una coordenada. caerColumna baja las fichas de una columna y rellena arriba. resolverCombinaciones recorre el ciclo de marcar, borrar y caer. insertarFila e insertarColumna reciben la posición, actualizan F o C y devuelven el puntero nuevo. eliminarFila y eliminarColumna compactan y, si hace falta, pasan por revisarMemoria. El encabezado no publica las funciones auxiliares del .cpp.

tablero.cpp incluye tablero.h, bits.h e iostream. Fuera de lo publicado en el encabezado, define marcarGrupos, recorrerGrupo, contarCombinaciones, borrarMarcadas, caerTablero, limpiarSobra y revisarMemoria. Las marcas y el arreglo de visitados se piden con new[] dentro de la función que los usa y se liberan ahí mismo con delete[]. mostrarBinario imprime cada byte del bit 7 al 0 bajo el título Tablero de bits. mostrarTablero recorre i = fila*C+col, traduce 0 a 5 a un char de un arreglo de tamaño 6 y, si la casilla está vacía, imprime un punto. Insertar siempre hace new[] y copia con leerFicha y escribirFicha. Eliminar compacta sobre el mismo arreglo y solo pide un bloque más pequeño cuando usadas*100 < reserva*65.

3.4 main.cpp

main.cpp no tiene un .h propio. Incluye iostream, bits.h y tablero.h. Imprime el nombre Sweet Crush, pide F y C, llama iniciarAzar y crearTablero, y entra en un while true. El menú oculta las opciones que no aplican si F o C llegan a cero. Cada opción válida ejecuta una sola acción, llama a resolverCombinaciones, acumula el puntaje y abre un bloque JUGADA n con la acción realizada, la estadística y las dos vistas. La opción 6 sale del ciclo y liberarTablero hace delete[] del bloque. Las funciones imprimirTitulo, lineaGruesa, mostrarAccion y mostrarEstadistica viven en este archivo porque solo sirven a la consola; no forman parte de tablero.h.

4. Esquema de tareas

Se armó por capas, para poder probar cada avance. Primero el empaquetado y las dos impresiones. Después borrar una casilla y hacer caer esa columna. Luego grupos de tres o más, borrarlos, caer el tablero y repetir. Con eso se metieron la inserción y la eliminación en cualquier posición, el 65% y el menú. En ejecución cada jugada entra por el menú, hace una sola de esas acciones, llama a resolverCombinaciones cuando toca, imprime los indicadores y vuelve a mostrar la trama y el tablero. La consola no se limpia: el menú recuerda la jugada previa y el reporte nuevo lleva el número de jugada, para no confundir un tablero con el otro al subir la ventana.

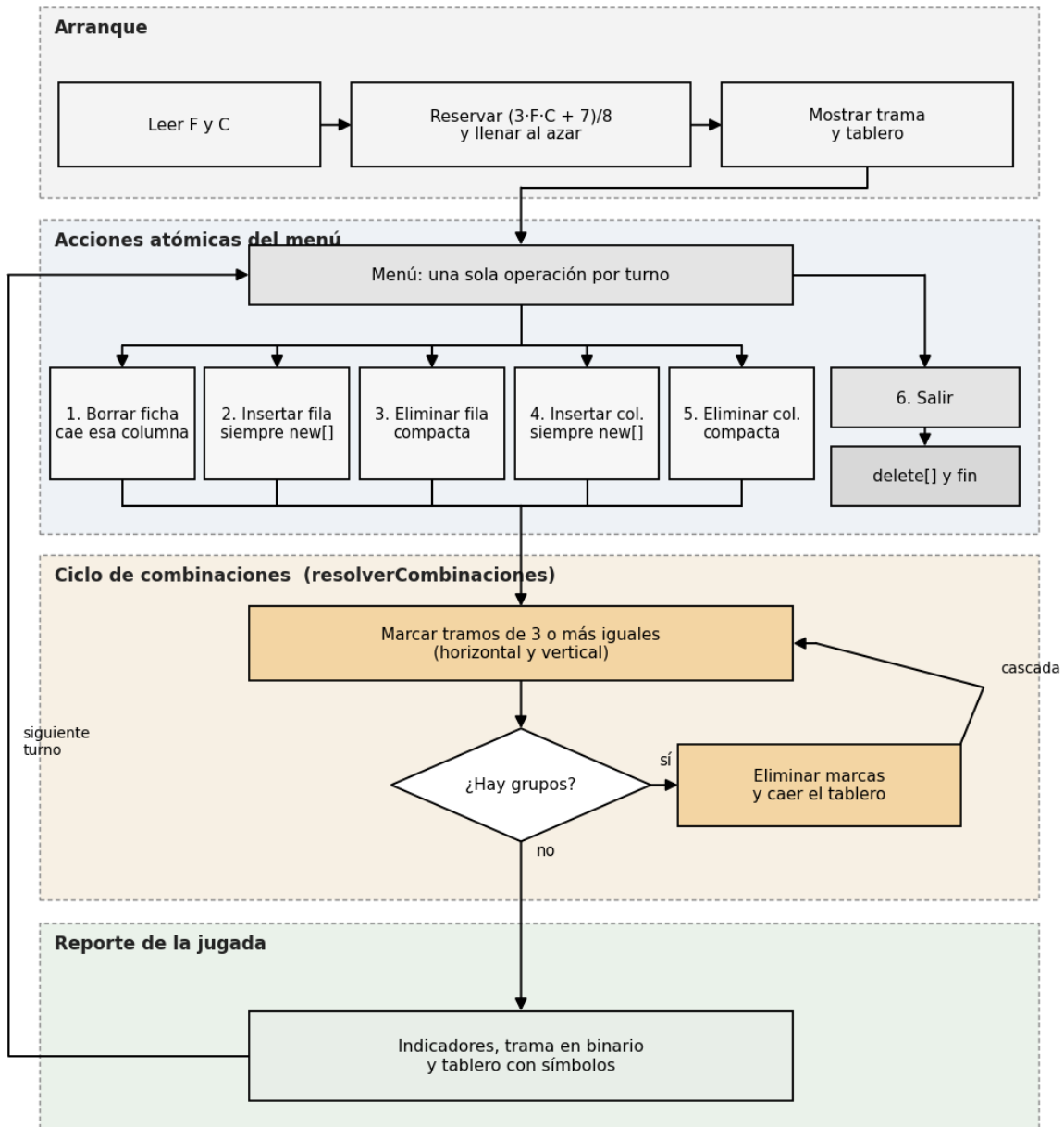


Figura 3. Esquema de tareas: arranque, acciones atómicas del menú, ciclo de combinaciones y reporte.

5. Algoritmos implementados

5.1 Lectura y escritura de tres bits

leerFicha recibe el índice lineal y calcula $\text{bitInicio} = \text{indice} * 3$, $\text{nByte} = \text{bitInicio} / 8$ y $\text{offset} = \text{bitInicio} \% 8$. Si desde el offset todavía caben tres bits, el valor es $(\text{datos}[\text{nByte}] \gg \text{offset}) \& 7$; el 7 corresponde a 111 en binario y deja pasar únicamente esos tres. Cuando la ficha se parte, del byte actual salen $8 - \text{offset}$ bits y del siguiente el resto, otra vez desde el LSB, y se recombinan con un OR. escribirFicha recorta el valor con $\& 7$, apaga los tres bits de destino con la máscara complementada y escribe el número nuevo. Si se omite la máscara, el corrimiento modifica la ficha adyacente.

```

unsigned char leerFicha(unsigned char* datos, int indice)
{
    int bitInicio = indice * 3;
    int nByte = bitInicio / 8;
    int offset = bitInicio % 8;
    if (offset + 3 <= 8) {
        return (datos[nByte] >> offset) & 7;
    }
    int bitsAqui = 8 - offset;
    unsigned char mascaraAqui = (1 << bitsAqui) - 1;
    unsigned char parteBaja = (datos[nByte] >> offset) & mascaraAqui;
    int bitsAlla = 3 - bitsAqui;
    unsigned char mascaraAlla = (1 << bitsAlla) - 1;
    unsigned char parteAlta = datos[nByte + 1] & mascaraAlla;
    return parteBaja | (parteAlta << bitsAqui);
}

```

Listado 1. Extracción de una ficha, incluido el caso en que se parte entre dos bytes (bits.cpp).

```

void escribirFicha(unsigned char* datos, int indice, unsigned char valor)
{
    valor = valor & 7;
    int bitInicio = indice * 3;
    int nByte = bitInicio / 8;
    int offset = bitInicio % 8;
    if (offset + 3 <= 8) {
        unsigned char mascara = 7 << offset;
        datos[nByte] = (datos[nByte] & ~mascara) | (valor << offset);
        return;
    }
    int bitsAqui = 8 - offset;
    unsigned char mascaraAqui = ((1 << bitsAqui) - 1) << offset;
    unsigned char parteBaja = valor & ((1 << bitsAqui) - 1);
    datos[nByte] = (datos[nByte] & ~mascaraAqui) | (parteBaja << offset);
    int bitsAlla = 3 - bitsAqui;
    unsigned char mascaraAlla = (1 << bitsAlla) - 1;
    unsigned char parteAlta = valor >> bitsAqui;
    datos[nByte + 1] = (datos[nByte + 1] & ~mascaraAlla) | (parteAlta & mascaraAlla);
}

```

Listado 2. Escritura de tres bits sin alterar las fichas adyacentes (bits.cpp).

5.2 Reserva inicial y vistas

bytesDelTablero calcula el techo $(3 \cdot F \cdot C + 7) / 8$. crearTablero pide ese tamaño, inicializa el bloque en ceros para que los bits sobrantes queden apagados a la izquierda, y llena cada casilla con fichaAlAzar. mostrarBinario imprime cada byte del bit 7 al 0. mostrarTablero recorre $i = \text{fila} \cdot C + \text{col}$, traduce los valores 0 a 5 al símbolo correspondiente y, si la casilla está vacía, imprime un punto.

```

int bytesDelTablero(int F, int C)
{
    return (3 * F * C + 7) / 8;
}

void iniciarAzar()
{
    srand(time(nullptr));
}

int fichaAlAzar()
{
    return rand() % 6;
}

```

5.3 Caída de una columna

caerColumna recorre la columna desde F-1 hacia arriba. Cada ficha distinta de 110 se escribe en la posición más baja aún libre, representada por la variable destino, que decrece cada vez que se pega una ficha. Las filas que quedan entre 0 y destino se completan al azar. Esta rutina no incrementa el contador de cascadas. Ese contador solo aumenta cuando la caída proviene de haber eliminado combinaciones.

```
void caerColumna(unsigned char* datos, int F, int C, int col)
{
    if (F < 1) {
        return;
    }
    int destino = F - 1;
    for (int fila = F - 1; fila >= 0; fila--) {
        unsigned char ficha = leerFicha(datos, fila * C + col);
        if (ficha != FICHA_VACIA) {
            escribirFicha(datos, destino * C + col, ficha);
            destino--;
        }
    }
    for (int fila = 0; fila <= destino; fila++) {
        escribirFicha(datos, fila * C + col, fichaAlAzar());
    }
}
```

Listado 4. Compactación vertical de una columna y relleno superior (tablero.cpp).

5.4 Combinaciones y cascada

marcarGrupos recorre cada fila y cada columna, mide rachas del mismo tipo entre 0 y 5 y, si la racha tiene longitud al menos 3, marca esas casillas. resolverCombinaciones entra en un ciclo: marca el tablero, termina si no hay marcas, y en caso contrario cuenta los grupos conexos, escribe 110 en las casillas marcadas, hace caer el tablero y suma una cascada. Los estados intermedios no se imprimen. Para el conteo, cada marca no visitada abre un grupo y recorrerGrupo se expande en las cuatro direcciones sobre casillas ya marcadas. De ese modo una T o una L quedan en un solo grupo, porque el cruce comparte ficha, y dos tiras disjuntas quedan en dos.

```
void resolverCombinaciones(unsigned char* datos, int F, int C,
                           int* nCombos, int* nCascadas, int* nFichas)
{
    *nCombos = 0;
    *nCascadas = 0;
    *nFichas = 0;
    if (F < 1 || C < 1) {
        return;
    }
    int n = F * C;
    while (true) {
        unsigned char* marcas = new unsigned char[n];
        marcarGrupos(datos, marcas, F, C);
```

```

    int hay = 0;
    for (int i = 0; i < n; i++) {
        if (marcas[i] != 0) { hay = 1; break; }
    }
    if (hay == 0) {
        delete[] marcas;
        break;
    }
    *nCombos = *nCombos + contarCombinaciones(marcas, F, C);
    *nFichas = *nFichas + borrarMarcadas(datos, marcas, F, C);
    delete[] marcas;
    caerTablero(datos, F, C);
    *nCascadas = *nCascadas + 1;
}
}

```

Listado 5. Ciclo de detección, borrado simultáneo, caída y conteo de cascadas (tablero.cpp).

5.5 Inserción y eliminación de filas o columnas

insertarFila calcula $F2 = F+1$, reserva el bloque nuevo, copia las filas anteriores a pos, escribe una fila aleatoria en pos y copia el resto una fila más abajo. insertarColumna procede de manera equivalente en el otro eje, usando $C2 = C+1$ en el destino. En ambos casos se actualizan nBytes y celdasReserva al producto de las dimensiones nuevas y se libera el puntero previo con delete[]. El parámetro pos puede ser F o C para agregar al final.

```

unsigned char* insertarFila(unsigned char* datos, int* F, int C, int pos,
                           int* nBytes, int* celdasReserva)
{
    int F2 = *F + 1;
    int nNuevo = bytesDelTablero(F2, C);
    unsigned char* nuevo = new unsigned char[nNuevo];
    for (int i = 0; i < nNuevo; i++) {
        nuevo[i] = 0;
    }
    for (int fila = 0; fila < pos; fila++) {
        for (int col = 0; col < C; col++) {
            escribirFicha(nuevo, fila * C + col,
                          leerFicha(datos, fila * C + col));
        }
    }
    for (int col = 0; col < C; col++) {
        escribirFicha(nuevo, pos * C + col, fichaAlAzar());
    }
    for (int fila = pos; fila < *F; fila++) {
        for (int col = 0; col < C; col++) {
            escribirFicha(nuevo, (fila + 1) * C + col,
                          leerFicha(datos, fila * C + col));
        }
    }
    delete[] datos;
    *F = F2;
    *nBytes = nNuevo;
    *celdasReserva = F2 * C;
    return nuevo;
}

```

Listado 6. Inserción de una fila: reserva nueva y copia ficha a ficha (tablero.cpp).

eliminarFila compacta sobre el mismo puntero: dest comienza en pos·C y se copian las fichas desde (pos+1)·C hasta el final. Se resta 1 a F, se apagan los bits que ya no corresponden a una ficha

mediante limpiarSobra y se llama a revisarMemoria. eliminarColumna recorre todas las casillas, omite la columna pos y escribe el resto en orden. En esa operación las fichas no descienden.

```
unsigned char* eliminarFila(unsigned char* datos, int* F, int C, int pos,
                           int* nBytes, int* celdasReserva)
{
    int dest = pos * C;
    int nOld = (*F) * C;
    for (int i = (pos + 1) * C; i < nOld; i++) {
        escribirFicha(datos, dest, leerFicha(datos, i));
        dest++;
    }
    *F = *F - 1;
    limpiarSobra(datos, *F, C, *nBytes);
    return revisarMemoria(datos, *F, C, nBytes, celdasReserva);
}
```

Listado 7. Eliminación de una fila con compactación in situ (tablero.cpp).

5.6 Criterio del 65 por ciento

La variable celdasReserva almacena cuántas celdas había cuando se pidió el bloque vigente. usadas es F·C. Si usadas·100 es mayor o igual que reserva·65, se conserva el puntero. Si es menor, se reserva el mínimo de bytes para las dimensiones actuales, se copian las fichas y se libera el bloque grande. La comparación se expresa en enteros para evitar el redondeo de usadas/reserva < 0.65 con punto flotante. En un tablero 10×10 se reservan 100 celdas y 38 bytes. Al quitar tres filas la ocupación es 70/100 y el bloque permanece en 38 bytes. Al quitar la cuarta, 60/100, el bloque pasa a $(180+7)/8 = 23$ bytes. Insertar una fila sobre ese 6×10 produce 7×10 y 27 bytes, de nuevo al 100% de la reserva nueva.

```
unsigned char* revisarMemoria(unsigned char* datos, int F, int C,
                              int* nBytes, int* celdasReserva)
{
    int usadas = F * C;
    if (*celdasReserva <= 0) {
        return datos;
    }
    if (usadas * 100 >= *celdasReserva * 65) {
        return datos;
    }
    int nNuevo = bytesDelTablero(F, C);
    unsigned char* nuevo = new unsigned char[nNuevo];
    for (int i = 0; i < nNuevo; i++) {
        nuevo[i] = 0;
    }
    for (int i = 0; i < usadas; i++) {
        escribirFicha(nuevo, i, leerFicha(datos, i));
    }
    delete[] datos;
    *nBytes = nNuevo;
    *celdasReserva = usadas;
    return nuevo;
}
```

Listado 8. Liberación del bloque solo cuando la ocupación baja de 0.65 (tablero.cpp).

tablero 10x10: 100 celdas, $(300+7)/8 = 38$ bytes

quitar 3 filas -> $70/100 = 70\%$

sigue el mismo bloque de 38 bytes

quitar la 4a fila -> $60/100 = 60\%$

new de $(180+7)/8 = 23$ bytes

Insertar fila o columna siempre pide un bloque nuevo y copia ficha a ficha.

Quitar compacta en el mismo arreglo. Ocupacion = $(F \times C \text{ actuales}) / \text{celdas de la ultima reserva}$.

Figura 4. Compactación in situ frente a new de un bloque más chico según el 65%.

5.7 Menú, indicadores y puntaje

Al arrancar se imprime Sweet Crush y se piden las filas y las columnas. Después de cada jugada válida se abre un bloque JUGADA n, se escribe qué se hizo (coordenada borrada o posición insertada o quitada) y salen F, C, bytes, ocupación, eliminaciones del usuario (1 si se borró una ficha y 0 si no), combinaciones, cascadas, fichas y dos puntajes: el de la jugada, igual al número de combinaciones, y el acumulado. Siguen el tablero de bits y el tablero del juego. Si se borró una ficha, fichasJugada vale 1 más las de combinaciones. Si se insertó o se quitó una fila o una columna, solo se cuentan las de combinaciones. Insertar no suma cascada por insertar; si después hay grupos y el tablero cae, esas caídas sí se cuentan. Cuando no quedan filas, el menú deja de ofrecer quitar fila y borrar ficha, y se puede insertar de nuevo.

```
nJugada = nJugada + 1;
cout << "          JUGADA " << nJugada << endl;
cout << "Que se hizo: ";
mostrarAccion(op, accionA, accionB);
mostrarEstadistica(F, C, nBytes, celdasReserva,
                  elimUsuario, combos, cascadas,
                  fichasJugada, puntajeTotal, 1);
mostrarBinario(datos, nBytes);
mostrarTablero(datos, F, C);
```

Listado 9. Separación de la jugada, la estadística y las dos vistas (main.cpp).

6. Problemas de desarrollo

En una ventana estrecha la trama en binario se parte en dos renglones y un byte puede parecer incompleto; conviene leer el identificador de byte y no la línea cortada.

Al insertar en una posición intermedia se consideró copiar el arreglo de bytes de una sola vez. En un tablero 2x3 la ficha de índice 2 ya está partida entre el byte 0 y el 1. Si se inserta una columna en la posición 1, los índices a la derecha se desplazan y los grupos de tres bits no coinciden con las mismas posiciones de byte, de modo que memcpy resulta incorrecto. La copia quedó expresada en llamadas a leerFicha y escribirFicha.

Si el 0.65 se escribe como división en punto flotante, una ocupación del 70% puede salir mal por redondeo. La desigualdad quedó $usadas * 100 < reserva * 65$ y con eso salen las cifras del 10×10. El conteo de combinaciones también se corrigió: si se suman por separado los tramos horizontales y los verticales, una T o una L cuentan como dos. Recorrer las marcas como un solo grupo deja el cruce en una combinación y dos tiras que no se tocan en dos. `eliminarColumna` no llama a `caerColumna`, porque compactar no es una cascada.

7. Evolución de la solución y consideraciones de implementación

El repositorio se construyó por avances. El primero dejó el tablero empaquetado y las dos vistas. El segundo permitió borrar una casilla, hacer caer esa columna y resolver combinaciones. El tercero incorporó la inserción y la eliminación de filas y columnas, `revisarMemoria`, el menú y el puntaje acumulado. El cuarto cubre el menú cuando el tablero se queda sin filas o sin columnas: no se ofrece borrar ficha ni quitar, y un tablero 0 no intenta caer ni resolver combinaciones. También queda el 1×1. El archivo de partida previsto en el diseño preliminar no se creó, porque el menú no introduce un estado distinto del tablero. El código quedó en `bits.h`, `bits.cpp`, `tablero.h`, `tablero.cpp`, `main.cpp` y `SweetCrush.pro`.

Los tipos siguen lo que guarda cada variable. El bloque es `unsigned char*` porque cada celda del arreglo es un byte de la trama, no una ficha. Un `int*` gastaría cuatro bytes por posición de memoria y no ayuda a empaquetar. La ficha también es `unsigned char`, recortada con 7: el valor cabe en tres bits. F, C, índices y contadores son `int`; son tamaños y posiciones, no el contenido de una casilla. `malloc` y `printf` no entran: las restricciones piden `new[]` / `delete[]` y `cin/cout`. Las marcas se piden dentro de la función que las usa y se sueltan ahí, para no dejar un `new` suelto. Si las filas llegan a cero, el menú deja de ofrecer quitar fila y borrar ficha, y se puede insertar de nuevo. El programa no cierra la partida solo. La consola no se borra: el bloque JUGADA n sirve para comparar la trama de antes con la de después.

Lo que más costó fue la ficha partida (offset 6 o 7), insertar en el medio sin copiar el arreglo de bytes, el 65% con enteros y no contar dos veces una T o una L. En Qt hay que marcar Run in terminal; si no, `cin` no lee. Quitar una columna no llama a `caerColumna`.

8. Conclusiones

Quedó un Sweet Crush de consola con el tablero en `unsigned char*` a tres bits por casilla, sin matriz auxiliar y con `new[]` / `delete[]`. Insertar fila o columna siempre pide un bloque nuevo y copia ficha a ficha. Quitar compacta en el mismo arreglo y solo reduce la reserva si la ocupación baja del

65%. El puntaje es un punto por combinación; una T o una L cuentan como una. El menú hace una sola operación por turno y, si el tablero se vacía, deja de ofrecer lo que ya no aplica.